

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

THIAGO MIORANDO TAVOLARA - 00335515

**METODOLOGIA PARA CONTROLE EM
TEMPO REAL COM FREERTOS EM
SISTEMAS EMBARCADOS APLICADA A
PLANTAS DIDÁTICAS.**

Porto Alegre
2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

THIAGO MIORANDO TAVOLARA - 00335515

**METODOLOGIA PARA CONTROLE EM
TEMPO REAL COM FREERTOS EM
SISTEMAS EMBARCADOS APLICADA A
PLANTAS DIDÁTICAS.**

Proposta de Trabalho de Conclusão de Curso
submetida à COMGRAD/CCA da UFRGS como parte
dos requisitos para a obtenção do título de *Bacharel
em Engenharia de Controle e Automação*.

Orientador:
Prof. Dr. Marcelo Götz

Porto Alegre
2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
ESCOLA DE ENGENHARIA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

THIAGO MIORANDO TAVOLARA - 00335515

**METODOLOGIA PARA CONTROLE EM
TEMPO REAL COM FREERTOS EM
SISTEMAS EMBARCADOS APLICADA A
PLANTAS DIDÁTICAS.**

Esta Proposta de Trabalho de Conclusão de Curso foi julgada adequada para a obtenção dos créditos da Atividade de *Trabalho de Conclusão de Curso CCA - I* e aprovada em sua forma final pelo Orientador e Banca Examinadora abaixo.

Orientador:

Prof. Dr. Marcelo Götz, UFRGS

Doutor pela Universität Paderborn – Paderborn, Alemanha
e pela Universidade Federal do Rio Grande do Sul – Porto
Alegre, Brasil

Banca Examinadora:

Prof. Dr. (nome do professor I), sigla da Instituição I onde atua
Doutor pela (Instituição Ia onde obteve o título – Cidade, País)

Profa. Dra. (nome da professora II), sigla da Instituição I onde atua
Doutor pela (Instituição IIa onde obteve o título – Cidade, País)

Fabiano Disconzi Wildner
Coordenador de Curso
Eng. de Controle e Automação

Porto Alegre
Maio - 2026

DEDICATÓRIA

Dedico este trabalho aos meus pais, em especial pela dedicação e apoio em todos os momentos difíceis.

AGRADECIMENTOS

À Universidade Federal do Rio Grande do Sul, UFRGS, pela oportunidade de realização de estudos.

Aos colegas de curso pelo seu auxílio nas tarefas desenvolvidas durante o curso e apoio na revisão deste trabalho.

Agradeço ao L^AT_EX por não ter vírus de macro. . .

RESUMO

Este documento foi criado com o objetivo de auxiliar aos alunos de graduação da Universidade Federal do Rio Grande do Sul (UFRGS) no desenvolvimento dos volumes finais de seus trabalhos de conclusão de curso. O presente documento serve, ao mesmo tempo, como modelo do formato de apresentação oficial do TCC-CCA, bem como de roteiro para as etapas de elaboração do texto técnico que o compõe. Os modelos e formatações aqui contidos são baseados em normas da Associação Brasileira de Normas Técnicas (ABNT), em especial a NBR-14724 (**ABNT:NBR-14724-2011**), que descreve o procedimento de elaboração e apresentação de trabalhos acadêmicos (dissertações, teses, monografias, entre outros).

Palavras-chave: Automação e Controle, Eletrônica e Instrumentação, Engenharia Elétrica, Processamento de Sinais

ABSTRACT

This document aims to support undergraduated students of Universidade Federal do Rio Grande do Sul (UFRGS) in the development of their diplom project relatory. This document serves concomitantly as a model of the official TCC-CCA presentation layout as well as a guide throughout technical text composition steps. The model and format held here are based in norms of the Associação Brasileira de Normas Técnicas (ABNT), mainly in NBR-14724 (**ABNT:NBR-14724-2011**) which defines guidelines of design and presentation of academic documents.

Palavras-chave: Automation and Control, Electrical Engineering, Electronic and Instrumentation, Signal Processing

LISTA DE ILUSTRAÇÕES

1	Diagrama de um sistema em malha aberta	17
2	Diagrama de um sistema em malha fechada	18
3	Diagrama de um sistema de controle com controlador PID	19
4	Diagrama de um sistema com controle liga-desliga (bang-bang)	20
5	Estrutura de múltiplas tarefas compartilhando o processador.	21
6	Estados das tarefas em um sistema multitarefa.	22
7	Vista geral da planta térmica didática HVAC utilizada nos experimentos.	27
8	Arquitetura eletrônica da planta térmica didática HVAC.	28
9	Interface gráfica preliminar desenvolvida em MATLAB para supervisão da planta térmica HVAC.	29

LISTA DE TABELAS

1	Parâmetros configuráveis na interface gráfica.....	30
2	Campos do protocolo serial CSV (Arduino → MATLAB).....	32
3	Definição das tarefas FreeRTOS e níveis de prioridade.....	33
4	Estimativa de uso de SRAM (ATmega328P — 2 KB total).	33

LISTA DE LISTAGENS

LISTA DE ABREVIATURAS

RTOS	Real Time Operating System
PID	Proporcional-Integral-Derivativo
FIFO	First In, First Out
ISR	Interrupt Service Routine

LISTA DE SÍMBOLOS

Σ Somatório

$\alpha\beta\pi\omega$ Fator de inconstância do resultado

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Objetivos	16
1.1.1	Objetivo Geral	16
1.1.2	Objetivos Específicos	16
2	FUNDAMENTAÇÃO TEÓRICA	17
2.1	Sistemas em Malha Aberta e Malha Fechada	17
2.2	Estratégias de Controle	17
2.2.1	Controlador PID	17
2.2.2	Controle Liga-Desliga (Bang-Bang)	19
2.3	Sistemas Operacionais de Tempo Real (RTOS)	19
2.4	FreeRTOS e Microcontroladores	20
2.4.1	Gerenciamento de Tarefas e Estados	21
2.4.2	Mecanismos de Escalonamento	22
2.4.3	Jitter Temporal e Determinismo	23
2.4.4	Timers, Interrupções e Controle de Recursos	23
2.4.5	Comunicação e Sincronização entre Tarefas	23
2.4.6	Restrições de Memória no ATmega328P	24
2.4.7	Integração e Robustez no Controle	24
2.5	Integração entre Arduino e Ambiente MATLAB	25
2.5.1	Integração entre Arduino e Ambiente MATLAB	25
3	METODOLOGIA	27
3.1	Descrição da Planta Térmica Didática	27
3.1.1	Arquitetura de Hardware e Componentes Utilizados	27
3.1.2	Interface Gráfica de Supervisão	28
3.1.2.1	Barra Superior	29
3.1.2.2	Painel de Parâmetros	29
3.1.2.3	Indicadores Numéricos	30
3.1.2.4	Área de Gráficos	30
3.1.3	Configuração do Ambiente de Desenvolvimento	31
3.1.3.1	Instalação das Bibliotecas e Dependências	31
3.1.3.2	Configuração da Comunicação Serial Arduino–MATLAB	31
3.1.4	Projeto da Arquitetura de Software Embarcado	32
3.1.4.1	Definição de Tarefas e Prioridades	32
3.1.4.2	Estratégia de Escalonamento Adotada	33
3.1.5	Implementação dos Algoritmos de Controle	34

4	PROPOSTA DE TRABALHO	35
4.1	Escopo e Delimitações	35
5	CRONOGRAMA DE ATIVIDADES	36
	REFERÊNCIAS	37

1 INTRODUÇÃO

Nas últimas décadas, o avanço tecnológico na área de sistemas embarcados, aliado ao crescimento da Internet das Coisas (IoT), tem impulsionado a demanda por sistemas cada vez mais complexos e capazes de interagir de forma eficiente com o mundo real. Dentro desse cenário, os sistemas de tempo real tornam-se essenciais, pois a validade do sistema não está relacionada apenas à exatidão dos resultados obtidos, mas também ao momento em que esses resultados são gerados (SPOHN; NETO; FASSINI, 2019). Vale ressaltar que não é obrigatório que a resposta a um evento ocorra no menor tempo possível, sendo mais importante que os prazos definidos para cada tarefa sejam rigorosamente atendidos (DENARDIN; BARRIQUELLO, 2019). Essa característica garante um comportamento mais determinístico do sistema, permitindo que as tarefas sejam concluídas dentro de prazos previamente estabelecidos, mesmo diante de variações nas condições de execução. Caso essas restrições temporais não sejam respeitadas, o sistema pode apresentar degradação de desempenho, inconsistências operacionais ou até falhas críticas, especialmente em aplicações onde confiabilidade e segurança são requisitos fundamentais.

Dentre as soluções de software disponíveis, o FreeRTOS (AMAZON WEB SERVICES, 2024) destaca-se como um kernel de tempo real amplamente utilizado em microcontroladores devido à sua robustez, modularidade e por ser um software de código aberto. Ele permite que a aplicação seja organizada como um conjunto de tarefas independentes, onde um escalonador decide qual tarefa deve ser executada com base em prioridades definidas pelo projetista (BARRY, 2016). Esse modelo oferece vantagens em relação às abordagens convencionais baseadas em laço principal (“superloop” ou bare-metal), que tendem a se tornar desorganizadas e menos eficientes à medida que a complexidade do sistema aumenta. Em aplicações multitarefa, a disputa por recursos pode resultar em atrasos indesejados e dificultar o controle adequado da execução das diferentes funções.

Para projetar e validar as estratégias de controle em sistemas dessa natureza, torna-se indispensável o uso de ambientes de computação técnica, como o MATLAB (THE MATHWORKS, INC., 2026). Essas plataformas permitem não apenas a modelagem e a simulação do comportamento dinâmico de sistemas complexos, mas também a integração direta com o hardware por meio do modo externo. Essa funcionalidade viabiliza o monitoramento de variáveis em tempo real, facilitando a análise de desempenho e o ajuste de parâmetros de controle diretamente no dispositivo embarcado, o que reduz o distanciamento entre o modelo matemático e a execução física.

No contexto acadêmico e de pesquisa, essa integração é frequentemente explorada por meio de plantas didáticas, que servem como elo entre a teoria de controle e a implementação prática. O controle de temperatura, especificamente, representa uma aplicação clássica e de alta relevância industrial, oferecendo dinâmicas ideais para validar a eficácia de algoritmos, como o PID, sob a gestão de um RTOS. O uso de tais plantas permite que desafios inerentes ao mundo real, como ruídos de sensores e latências de processamento, sejam enfrentados e mitigados de forma controlada.

Diante desse cenário, este trabalho tem como objetivo desenvolver uma metodologia para o controle em tempo real utilizando o FreeRTOS integrado ao ambiente MATLAB, aplicada a uma planta térmica didática. A proposta busca não apenas validar a arquitetura de software desenvolvida, mas também analisar de que forma o escalonamento de tarefas impacta o desempenho do processo. Com isso, pretende-se consolidar um guia prático para a implementação de sistemas de controle multitarefa em plataformas embarcadas, garantindo o rigor temporal e a eficiência operacional.

1.1 OBJETIVOS

1.1.1 Objetivo Geral

Desenvolver e validar uma arquitetura de controle em tempo real para sistemas embarcados, utilizando o sistema operacional FreeRTOS, com integração a ferramentas de supervisão e análise no ambiente MATLAB, visando aplicações em plantas didáticas.

1.1.2 Objetivos Específicos

- Implementar e gerenciar tarefas concorrentes em um microcontrolador utilizando o FreeRTOS, considerando aspectos de escalonamento e temporização.
- Desenvolver e implementar algoritmos de controle adequados para execução em tempo real em sistemas embarcados.
- Integrar o sistema embarcado ao ambiente MATLAB para fins de monitoramento, aquisição de dados e análise do comportamento dinâmico.
- Validar experimentalmente a arquitetura proposta por meio de sua aplicação em uma planta térmica didática.
- Analisar a influência do escalonamento de tarefas no desempenho do sistema de controle.

2 FUNDAMENTAÇÃO TEÓRICA

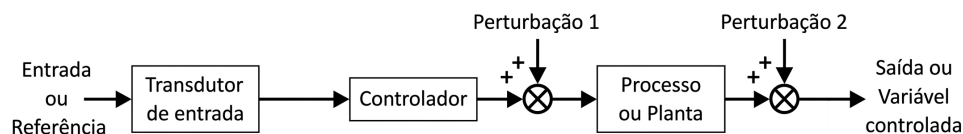
Este capítulo estabelece os pilares teóricos necessários para a compreensão e o desenvolvimento do sistema de controle térmico sob um sistema operacional de tempo real. A fundamentação abrange desde a teoria de controle clássica até as especificidades arquiteturais do FreeRTOS e sua integração com ferramentas de computação técnica.

2.1 SISTEMAS EM MALHA ABERTA E MALHA FECHADA

A classificação de sistemas de controle é fundamentada na topologia do fluxo de sinais. De acordo com Nise (2017), sistemas em malha aberta são aqueles cuja ação de controle independe da saída do processo, conforme ilustrado na Figura 1. Embora apresentem baixa complexidade e custo reduzido, esses sistemas são incapazes de compensar perturbações externas ou variações nos parâmetros internos da planta.

Em contrapartida, sistemas em malha fechada utilizam o princípio da realimentação (*feedback*). Segundo Ogata (2010), a malha fechada permite que a saída do sistema seja continuamente comparada a um valor de referência (*setpoint*), como apresentado na Figura 2. O erro resultante dessa comparação orienta o controlador a ajustar a variável manipulada, conferindo ao sistema maior precisão, robustez e capacidade de rejeição a distúrbios, ainda que exija um projeto de estabilidade mais rigoroso para evitar oscilações indesejadas.

Figura 1: Diagrama de um sistema em malha aberta

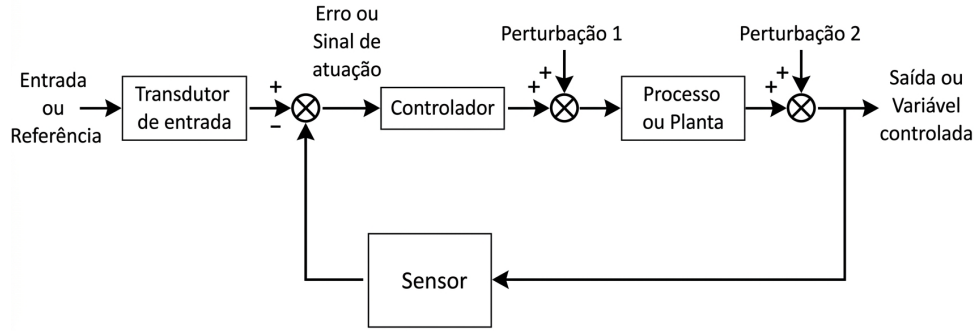


Fonte: Autor.

2.2 ESTRATÉGIAS DE CONTROLE

2.2.1 Controlador PID

O controlador Proporcional-Integral-Derivativo (PID) é uma das estratégias de controle mais utilizadas na automação industrial devido à sua simplicidade de implementação e elevada eficiência em diferentes tipos de processos. Segundo Júnior e Silva (2015), a

Figura 2: Diagrama de um sistema em malha fechada

Fonte: Autor.

ampla aplicação desse controlador está associada à sua capacidade de conduzir o sistema ao ponto de operação desejado de maneira rápida, reduzindo oscilações e minimizando o erro em regime permanente.

A ação de controle do PID é composta por três parcelas fundamentais:

- **Proporcional** (K_p): produz uma ação proporcional ao erro instantâneo, proporcionando resposta rápida às variações do sistema;
- **Integral** (K_i): acumula o erro ao longo do tempo, atuando na eliminação do erro em regime permanente;
- **Derivativo** (K_d): considera a taxa de variação do erro, antecipando tendências e contribuindo para o amortecimento da resposta do sistema.

Matematicamente, a ação de controle de um controlador PID pode ser descrita por:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

Uma forma alternativa, também amplamente empregada na literatura e em aplicações industriais, é apresentada pela Equação 2 (BAZANELLA; SILVA JR., 2005):

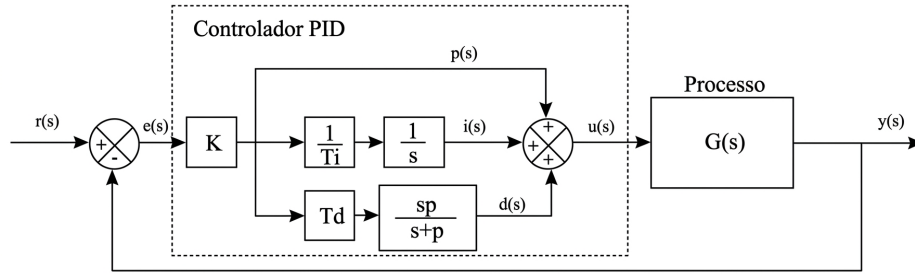
$$u(t) = K \left(e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \frac{de(t)}{dt} \right) \quad (2)$$

Nessa representação, K corresponde ao ganho proporcional do controlador, T_i representa a constante de tempo associada à ação integral e T_d a constante de tempo da ação derivativa. O termo $e(t)$ representa o erro entre o valor de referência e a saída do sistema. A adequada combinação desses parâmetros permite ajustar características importantes da resposta dinâmica, como tempo de subida, sobressinal, tempo de acomodação e erro em regime permanente.

Na prática, a sintonia dos parâmetros do controlador pode ser realizada por diferentes abordagens, desde métodos empíricos, como o método de Ziegler-Nichols, até técnicas analíticas e computacionais mais avançadas, dependendo da complexidade e das exigências do sistema. Um ajuste inadequado desses parâmetros pode resultar em oscilações excessivas, respostas lentas ou até mesmo instabilidade.

A Figura 3 apresenta a estrutura típica de um controlador PID baseado na representação mostrada na Equação 2, aplicada a um sistema de controle em malha fechada.

Figura 3: Diagrama de um sistema de controle com controlador PID



Fonte: (BAZANELLA; SILVA JR., 2005).

2.2.2 Controle Liga-Desliga (Bang-Bang)

Comum em sistemas de climatização e aquecimento simples, o controle liga-desliga opera de forma binária. O atuador é ativado ou desativado com base no sinal do erro, sem níveis intermediários de atuação. Esse tipo de estratégia é conhecido como controle de dois níveis ou *on-off control*, sendo amplamente utilizado devido à sua simplicidade e baixo custo (OGATA, 2010).

Nesse contexto, a ação bang-bang corresponde a um controlador cujo sinal de saída assume apenas dois valores distintos, conforme descrito por Bazanella e Gomes da Silva Jr. (2005). A lei de controle pode ser expressa de forma simplificada como:

$$u(t) = \begin{cases} U_{\max}, & \text{se } e(t) > 0 \\ U_{\min}, & \text{se } e(t) < 0 \end{cases} \quad (3)$$

onde $e(t)$ representa o erro entre o valor de referência e a variável de processo. Devido à natureza discreta da ação de controle, o sistema tende a apresentar oscilações em torno do *setpoint*, caracterizando um comportamento típico de ciclo limite (OGATA, 2010).

Uma forma comum de mitigar o efeito de chaveamento excessivo é a introdução de uma banda de histerese, criando uma região de tolerância em torno do *setpoint*. Essa abordagem reduz a frequência de comutação e evita chaveamentos indesejados na presença de ruído (BAZANELLA; SILVA JR., 2005).

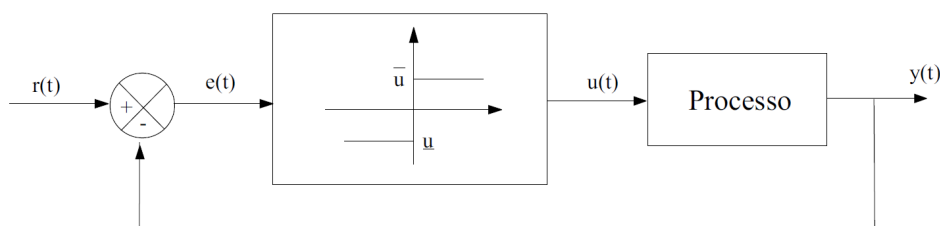
Apesar de suas limitações em termos de precisão e desempenho dinâmico, o controle liga-desliga permanece amplamente utilizado em aplicações onde pequenas oscilações são aceitáveis e a simplicidade é um fator determinante (OGATA, 2010).

A Figura 4 ilustra o comportamento típico de um controlador liga-desliga aplicado a um sistema de controle.

2.3 SISTEMAS OPERACIONAIS DE TEMPO REAL (RTOS)

Diferentemente dos sistemas operacionais de propósito geral, cujo objetivo principal é maximizar o desempenho médio e a interação com o usuário, um Sistema Operacional de Tempo Real (RTOS) é projetado para garantir previsibilidade temporal. Isso implica que não basta apenas que os resultados estejam logicamente corretos, mas que também sejam produzidos dentro de intervalos de tempo previamente estabelecidos. Conforme Laplante (2004), a validade de um sistema de tempo real depende diretamente do cumprimento

Figura 4: Diagrama de um sistema com controle liga-desliga (bang-bang)



Fonte: (BAZANELLA; SILVA JR., 2005).

rigoroso dos prazos (*deadlines*), sendo que uma resposta correta fora do tempo especificado pode ser considerada inválida.

Os sistemas de tempo real podem ser classificados quanto à rigidez de seus requisitos temporais em três categorias (SPOHN; NETO; FASSINI, 2019):

- **Hard:** o não cumprimento dos prazos inutiliza completamente o sistema;
- **Firm:** o resultado fora do prazo torna-se inútil, mas não compromete o sistema como um todo;
- **Soft:** o descumprimento dos prazos impacta apenas o desempenho, sem causar dano grave.

Nesse contexto, o RTOS oferece mecanismos essenciais para o desenvolvimento de aplicações críticas, como o gerenciamento de múltiplas tarefas concorrentes, sincronização entre processos e controle preciso de temporização. Denardin e Barriquello (2019) ressaltam que o uso de um RTOS permite organizar a aplicação em tarefas independentes, cada uma responsável por uma funcionalidade específica, como aquisição de dados de sensores, execução de algoritmos de controle ou comunicação com outros dispositivos. Essas tarefas são gerenciadas por um escalonador (*scheduler*), que determina qual tarefa deve ser executada a cada instante com base em critérios como prioridade e tempo de execução.

Dessa forma, o uso de um RTOS garante que requisitos fundamentais — como a manutenção de uma taxa de amostragem constante da planta térmica — sejam atendidos, contribuindo diretamente para a estabilidade e o desempenho do sistema de controle.

2.4 FREERTOS E MICROCONTROLADORES

O FreeRTOS é um sistema operacional de tempo real de código aberto, desenvolvido em linguagem C e projetado para aplicações embarcadas que demandam previsibilidade temporal e execução multitarefa em ambientes com recursos restritos (BARRY, 2016). No contexto do microcontrolador ATmega328P, o FreeRTOS substitui o modelo tradicional de laço único (*superloop*) por um núcleo (*kernel*) que gerencia tarefas independentes, conferindo maior modularidade e escalabilidade ao projeto (SPOHN; NETO; FASSINI, 2019).

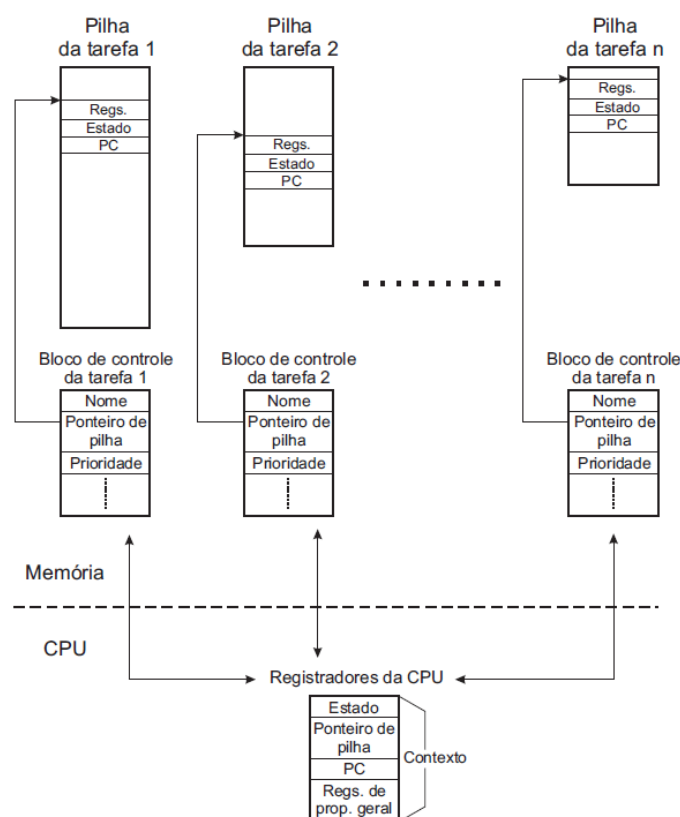
2.4.1 Gerenciamento de Tarefas e Estados

No FreeRTOS, as aplicações são organizadas como coleções de tarefas independentes, semelhantes a *threads*, cada uma possuindo sua própria pilha (*stack*) e um bloco de controle denominado *Task Control Block* (TCB) (SPOHN; NETO; FASSINI, 2019). Em processadores de núcleo único, como o ATmega328P, o escalonador promove uma rápida alternância entre tarefas, criando a ilusão de simultaneidade (*pseudo-parallelismo*).

O bloco de controle de tarefa (*Task Control Block* – TCB) armazena as informações necessárias para o gerenciamento de cada tarefa pelo escalonador do sistema operacional. Entre essas informações estão o ponteiro da pilha da tarefa, registradores salvos durante trocas de contexto, estado atual da tarefa e nível de prioridade. Conforme descrito por Denardin e Barriquello (2019), o TCB é essencial para que o RTOS consiga interromper e restaurar tarefas de forma transparente durante o escalonamento.

A Figura 5 apresenta uma representação simplificada da organização das tarefas em memória, destacando a relação entre as pilhas individuais, os TCBs e o processador.

Figura 5: Estrutura de múltiplas tarefas compartilhando o processador.



Fonte: (DENARDIN; BARRIQUELLO, 2019).

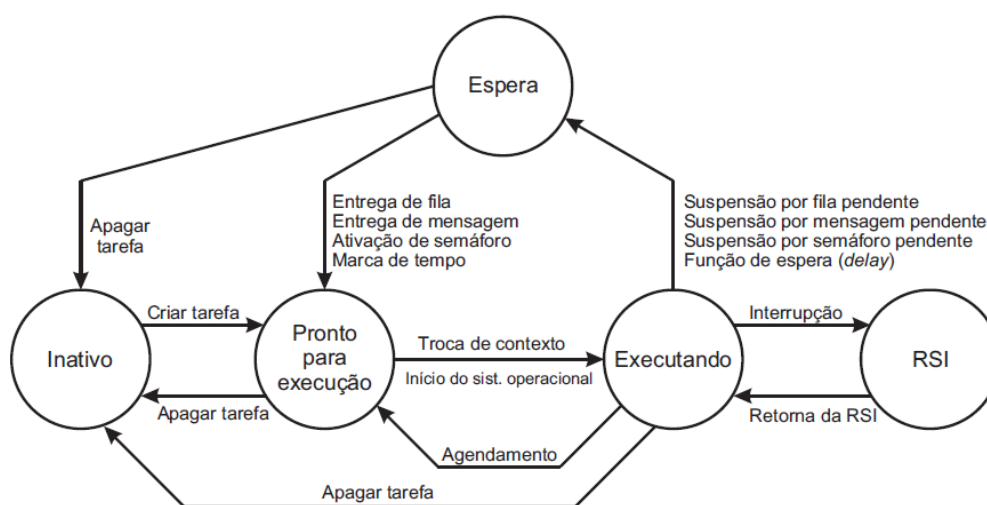
As tarefas transitam entre diferentes estados de execução ao longo do funcionamento do sistema operacional. Esses estados representam a situação atual da tarefa em relação ao uso do processador e aos mecanismos de sincronização do sistema. Os principais estados utilizados no FreeRTOS são:

- **Executando:** a tarefa está utilizando o processador naquele instante;

- **Pronta:** a tarefa está apta para execução, aguardando apenas a liberação da CPU pelo escalonador;
- **Bloqueada:** a tarefa aguarda um evento temporal ou de sincronização, como atraso periódico, recebimento de mensagem em fila ou liberação de semáforo;
- **Suspensa:** a tarefa foi removida explicitamente do escalonador, retornando apenas mediante chamada específica da API do FreeRTOS.

A Figura 6 ilustra os estados fundamentais das tarefas e as possíveis transições entre eles em um sistema multitarefa.

Figura 6: Estados das tarefas em um sistema multitarefa.



Fonte: (DENARDIN; BARRIQUELLO, 2019).

Durante a troca de contexto, o estado da tarefa atualmente em execução é salvo em sua pilha e no respectivo TCB, permitindo que outra tarefa utilize o processador. Quando a tarefa interrompida retorna à execução, seu contexto anterior é restaurado, garantindo continuidade transparente da aplicação (DENARDIN; BARRIQUELLO, 2019).

2.4.2 Mecanismos de Escalonamento

O escalonador decide qual tarefa utilizará a CPU com base em prioridades fixas definidas pelo desenvolvedor. Conforme Spohn, Neto e Fassini (2019), o sistema opera em dois modos principais:

- **Escalonamento preemptivo:** se uma tarefa de maior prioridade tornar-se pronta, ela interrompe imediatamente a tarefa atual (*troca de contexto*), reduzindo a latência e assegurando o determinismo exigido em sistemas de controle. Testes realizados na plataforma Arduino Uno demonstraram latência média de troca de contexto de apenas 7,74 µs (SPOHN; NETO; FASSINI, 2019);
- **Fatiamento de tempo (*time slicing*):** utiliza interrupções periódicas do sistema (*ticks*) para compartilhar o processador equitativamente entre tarefas de mesma prioridade.

Embora os testes reportados por Spohn, Neto e Fassini(2019) tenham sido realizados na plataforma Arduino Uno, os resultados são diretamente aplicáveis ao Arduino Nano utilizado neste trabalho, uma vez que ambas as plataformas utilizam o mesmo microcontrolador ATmega328P, operando à mesma frequência de clock de 16 MHz.

2.4.3 Jitter Temporal e Determinismo

Em sistemas de controle em tempo real, pequenas variações no instante de execução das tarefas periódicas são conhecidas como *jitter* temporal. Esse fenômeno ocorre devido a atrasos introduzidos pelo escalonador, interrupções concorrentes ou variações no tempo de execução das tarefas, podendo comprometer diretamente o desempenho de sistemas de controle digital.

Em aplicações embarcadas de controle, a manutenção de uma taxa de amostragem constante é fundamental para garantir estabilidade e previsibilidade da resposta dinâmica. Variações excessivas no período de amostragem podem alterar o comportamento do controlador discreto, produzindo oscilações indesejadas, degradação de desempenho ou até instabilidade do sistema.

Nesse contexto, o FreeRTOS oferece mecanismos para reduzir o *jitter* e aumentar o determinismo temporal da aplicação. Uma das principais estratégias consiste na utilização da função `vTaskDelayUntil()`, que permite a execução periódica de tarefas com base em referências absolutas de tempo, minimizando o acúmulo de atrasos ao longo das execuções sucessivas.

Além disso, o escalonamento preemptivo baseado em prioridades garante que tarefas críticas, como aquisição de sensores e execução do algoritmo de controle, tenham acesso prioritário ao processador sempre que necessário. Essa característica contribui diretamente para o comportamento determinístico exigido em aplicações de controle em tempo real.

2.4.4 Timers, Interrupções e Controle de Recursos

O FreeRTOS disponibiliza *timers* de software para agendamento de ações periódicas ou temporizadas sob controle do kernel, operando no modo *one-shot* (executado uma única vez) ou *auto-reload* (reiniciado automaticamente a cada período) (SPOHN; NETO; FASSINI, 2019). As rotinas de interrupção (*Interrupt Service Routines* – ISR) possuem prioridade superior às tarefas convencionais, garantindo menor latência no tratamento de eventos críticos; recomenda-se, contudo, que executem apenas operações rápidas, delegando o processamento mais complexo a tarefas do sistema.

2.4.5 Comunicação e Sincronização entre Tarefas

Em ambientes multitarefa, tarefas compartilham a memória global do sistema sem proteção automática, o que torna possível que uma tarefa seja interrompida durante a escrita de uma variável compartilhada por outra de maior prioridade. Esse cenário caracteriza as chamadas condições de corrida (*race conditions*), podendo gerar corrupção de dados e comportamentos não determinísticos — problema especialmente crítico em aplicações de controle em tempo real (DENARDIN; BARRIQUELLO, 2019).

Para prevenir tais situações, o FreeRTOS oferece um conjunto de mecanismos de comunicação e sincronização entre tarefas (*Interprocess Communication – IPC*) (BARRY, 2016):

- **Filas (*Queues*):** estruturas FIFO utilizadas para envio seguro de dados entre tarefas ou entre rotinas de interrupção e tarefas, permitindo a transferência organizada de mensagens sem risco de corrupção, mesmo sob acesso concorrente;
- **Semáforo Binário:** mecanismo de sincronização entre duas tarefas ou de sinalização de eventos simples, permitindo que uma tarefa aguarde a liberação de determinado recurso;
- **Semáforo Contador (*Counting Semaphore*):** extensão do semáforo binário que possibilita controlar múltiplas ocorrências simultâneas de um mesmo recurso compartilhado;
- **MUTEX (*Mutual Exclusion*):** mecanismo de exclusão mútua para proteção de regiões críticas. Diferentemente dos semáforos binários, implementa herança de prioridade, mitigando problemas de inversão de prioridade em sistemas multitarefa;
- **Notificações de Tarefa (*Task Notifications*):** mecanismo leve de sinalização implementado diretamente no TCB da tarefa, dispensando estruturas auxiliares e reduzindo o overhead de comunicação.

Em conjunto, esses mecanismos — somados às regiões críticas já mencionadas — garantem previsibilidade temporal, integridade dos dados e confiabilidade do sistema, pilares fundamentais de qualquer aplicação embarcada com múltiplas tarefas concorrentes.

2.4.6 Restrições de Memória no ATmega328P

A implementação do FreeRTOS no ATmega328P exige atenção rigorosa aos 2 KB de SRAM disponíveis. Conforme Spohn, Neto e Fassini (2019), o *port* para a arquitetura AVR utiliza o esquema **Heap 3**, que encapsula as funções `malloc()` e `free()` da biblioteca padrão; para garantir a segurança em ambiente multitarefa (*thread-safety*), o kernel suspende temporariamente o escalonador durante as alocações. Como a pilha de cada tarefa é alocada na SRAM, pilhas excessivamente grandes ou um número elevado de tarefas podem causar colisão de memória (*heap overflow*), resultando em instabilidades. Os testes mostram que é possível executar até 10 tarefas simultâneas, com pilhas de no máximo 89 bytes cada.

2.4.7 Integração e Robustez no Controle

O FreeRTOS atua como uma camada de abstração entre o hardware e a aplicação, permitindo que diferentes funcionalidades do sistema sejam organizadas em tarefas independentes e executadas de forma concorrente (MORAIS, 2018). Essa arquitetura favorece a modularidade do software, simplifica a manutenção do código e melhora a escalabilidade da aplicação embarcada.

No sistema desenvolvido neste trabalho, tarefas críticas — como aquisição do sensor de temperatura e execução do algoritmo de controle — possuem prioridade elevada, garantindo previsibilidade temporal e baixa latência de resposta. Em contrapartida, tarefas de menor criticidade, como transmissão serial de dados ao MATLAB, são executadas com prioridade reduzida, evitando interferência direta sobre o laço de controle.

A utilização do FreeRTOS também contribui para o aumento da robustez do sistema, uma vez que o isolamento funcional das tarefas reduz o impacto de falhas locais sobre toda a aplicação. Além disso, mecanismos de sincronização, gerenciamento de memória e controle de prioridades permitem maior confiabilidade na execução simultânea de múltiplas funções em sistemas embarcados de recursos limitados.

No contexto da planta térmica didática utilizada neste trabalho, o uso do FreeRTOS permite garantir periodicidade adequada para aquisição de dados, atualização do sinal de controle e comunicação supervisória, assegurando comportamento determinístico compatível com os requisitos de sistemas de controle em tempo real.

2.5 INTEGRAÇÃO ENTRE ARDUINO E AMBIENTE MATLAB

2.5.1 *Integração entre Arduino e Ambiente MATLAB*

A integração entre sistemas embarcados e ambientes de computação técnica representa uma abordagem consolidada no desenvolvimento e na validação de sistemas de controle, tanto em contextos acadêmicos quanto industriais. Plataformas como o MATLAB (THE MATHWORKS, INC., 2026) oferecem recursos para modelagem, simulação, aquisição de dados e análise do comportamento dinâmico de processos, complementando a execução embarcada ao fornecer uma camada de supervisão externa ao microcontrolador.

Neste trabalho, será utilizada a plataforma MATLAB R2014a, disponibilizada por meio da licença institucional da Universidade Federal do Rio Grande do Sul (UFRGS). A escolha dessa versão deve-se à sua disponibilidade institucional, estabilidade de operação e ampla compatibilidade com bibliotecas de comunicação serial utilizadas em sistemas embarcados.

Será desenvolvida uma Interface Gráfica do Usuário (GUI – Graphical User Interface) em ambiente MATLAB, destinada à supervisão e interação com a planta térmica controlada por um microcontrolador Arduino Nano. A interface permitirá a comunicação serial bidirecional entre o computador e o sistema embarcado, possibilitando o envio e o recebimento de parâmetros em tempo real.

Nesse arranjo, o Arduino Nano será responsável pela execução das tarefas de controle em tempo real, incluindo a aquisição dos sinais provenientes dos sensores, o cálculo das ações de controle e o acionamento dos atuadores presentes na planta térmica. Por outro lado, o MATLAB atuará como supervisor, realizando o monitoramento contínuo das variáveis do processo e permitindo a alteração dinâmica dos parâmetros de operação sem a necessidade de recompilação ou reinicialização do firmware.

A GUI desenvolvida possibilitará a visualização gráfica das variáveis do sistema, como temperatura medida, sinal de controle aplicado e valor de referência, além da seleção dos modos de operação da planta, incluindo estratégias de controle em malha aberta, Bang-Bang e PID. Também serão disponibilizados campos para parametrização dos ganhos de controle e indicadores numéricos referentes ao estado atual do sistema.

A comunicação entre os dois ambientes será estabelecida por meio de uma interface serial UART, devido à sua simplicidade de implementação e ampla compatibilidade com diferentes plataformas de hardware e software. O protocolo de comunicação será baseado em mensagens textuais estruturadas no formato CSV (Comma-Separated Values), permitindo a troca bidirecional de informações de maneira simples e eficiente. Nesse modelo, o Arduino Nano enviará continuamente os dados monitorados da planta térmica ao MATLAB, enquanto a GUI poderá transmitir novos parâmetros e comandos operacionais em tempo real.

No contexto de plantas didáticas, essa integração apresenta elevada relevância, pois permite que fenômenos inerentes ao ambiente físico — como ruídos de medição, atrasos de comunicação, saturação de atuadores e variações paramétricas — sejam observados e analisados diretamente durante a execução experimental. Dessa forma, a utilização da GUI supervisória em conjunto com o sistema embarcado contribui significativamente para a validação prática das estratégias de controle implementadas e para o processo de aprendizagem associado aos sistemas de controle embarcados.

3 METODOLOGIA

3.1 DESCRIÇÃO DA PLANTA TÉRMICA DIDÁTICA

A planta térmica didática utilizada neste trabalho é baseada no módulo HVAC (*Heating, Ventilation and Air Conditioning*) da Quanser, adaptado para aplicações de controle embarcado com microcontrolador ATmega328. O sistema tem como objetivo permitir a implementação, validação e análise de diferentes estratégias de controle térmico em tempo real, integrando hardware embarcado, supervisão computacional e algoritmos de controle.

A estrutura física da planta é composta por uma fonte de aquecimento, responsável pela elevação da temperatura interna, e por um sistema de ventilação forçada, utilizado para promover o resfriamento do ambiente controlado. A variável de processo monitorada corresponde à temperatura interna, medida por sensor digital, enquanto as variáveis manipuladas são os sinais de acionamento dos atuadores de aquecimento e ventilação.

A Figura 7 apresenta uma visão geral da planta física.

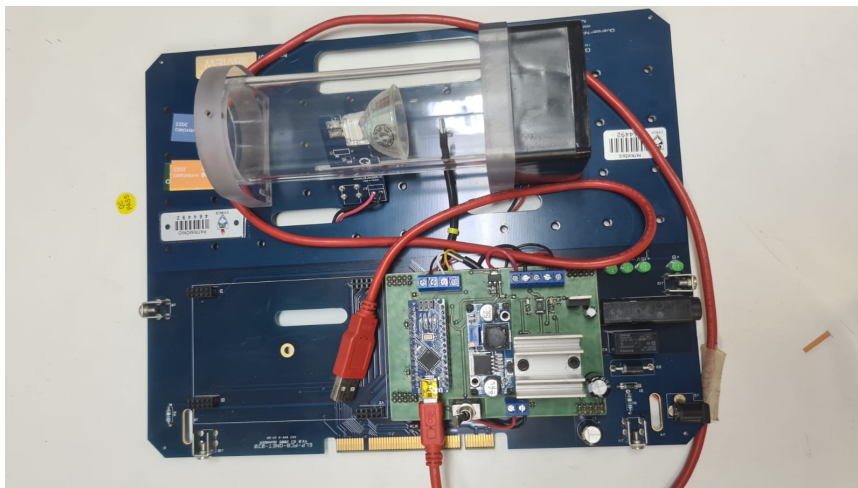


Figura 7: Vista geral da planta térmica didática HVAC utilizada nos experimentos.

Fonte: Autor.

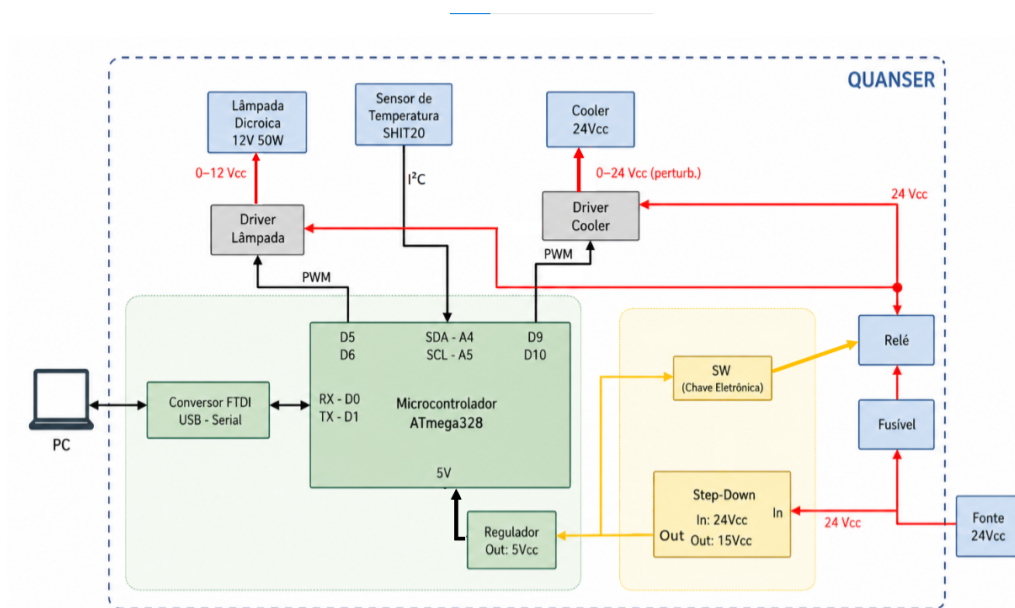
3.1.1 Arquitetura de Hardware e Componentes Utilizados

O sistema é centrado no microcontrolador ATmega328P presente no Arduino Nano, que realiza a aquisição de temperatura, executa os algoritmos de controle e aciona os

atuadores via PWM. A comunicação com o computador é feita pelo conversor USB-Serial integrado ao Arduino Nano (chip CH340G ou FT232RL, dependendo da versão da placa), sem necessidade de conversor externo. A Figura 8 ilustra o diagrama da arquitetura eletrônica do sistema, cujos principais componentes são descritos a seguir:

- **Controle e comunicação:** Arduino Nano com microcontrolador ATmega328P (aquisição, controle e acionamento) e conversor USB-Serial integrado (comunicação serial com o PC);
- **Sensoriamento:** sensor SHT20 via I²C (pinos A4/A5) para medição da temperatura interna;
- **Atuadores:** lâmpada dicróica 12 V/50 W para aquecimento, controlada por PWM nos pinos D5/D6, e cooler 24 Vcc para resfriamento, acionado por PWM nos pinos D9/D10;
- **Alimentação e proteção:** fonte de 24 Vcc como entrada principal, com conversão em cascata via Step-Down (24 V $\rightarrow$ 15 V) e regulador linear (15 V $\rightarrow$ 5 V), além de relé para chaveamento da carga e fusível para proteção contra sobrecorrente.

Figura 8: Arquitetura eletrônica da planta térmica didática HVAC.



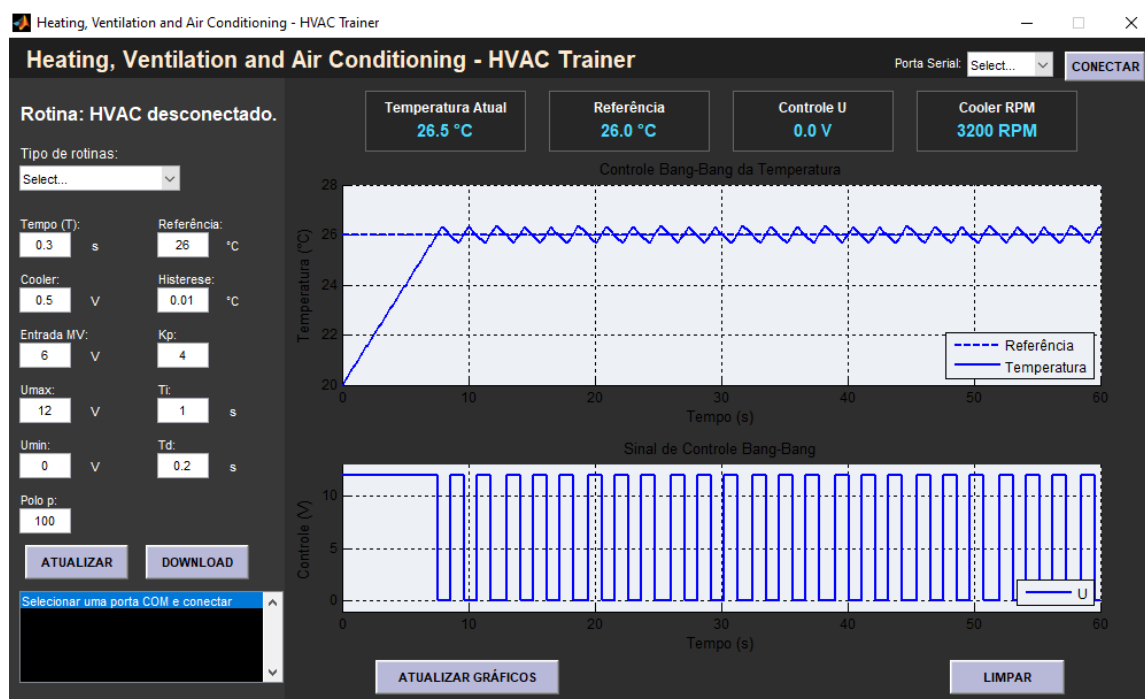
Fonte: Adaptado do Autor Marcos Vizzotto.

3.1.2 Interface Gráfica de Supervisão

Com o objetivo de facilitar a operação da planta e a visualização dos dados em tempo real, foi desenvolvida uma interface gráfica de supervisão (*Graphical User Interface* — GUI) no ambiente MATLAB R2014a. A interface foi implementada de forma programática, em um único arquivo **.m**, dispensando o uso do editor GUIDE e facilitando a manutenção e o versionamento do código.

A Figura ?? apresenta a tela principal da interface, cujos elementos são organizados em quatro regiões funcionais descritas a seguir.

Figura 9: Interface gráfica preliminar desenvolvida em MATLAB para supervisão da planta térmica HVAC.



Fonte: Autor.

3.1.2.1 Barra Superior

A barra superior concentra os controles de conexão serial. Um menu suspenso (*popupmenu*) permite selecionar a porta COM à qual o Arduino Nano está conectado, e o botão **CONECTAR** estabelece a comunicação serial a 57.600 baud por meio da função `serial()` do MATLAB. O título da janela identifica o sistema supervisionado: *Heating, Ventilation and Air Conditioning — HVAC Trainer*.

3.1.2.2 Painel de Parâmetros

O painel lateral esquerdo reúne todos os controles de configuração do experimento. Um segundo menu suspenso permite selecionar o modo de operação da planta entre as três estratégias implementadas: Malha Aberta, Bang-Bang e PID. Abaixo, onze campos de edição (*edit boxes*) agrupados em dois por linha permitem ajustar os parâmetros de controle descritos no Quadro ??.

O botão **ATUALIZAR** formata os valores dos campos em uma linha CSV e a envia ao Arduino pelo protocolo definido na Seção ??, permitindo a alteração de parâmetros em tempo real sem necessidade de reinicialização do sistema embarcado. O botão **DOWNLOAD** aciona o script `salvarCSV.m`, exportando os dados da sessão corrente para um arquivo com cabeçalho identificado por *timestamp*.

Tabela 1: Parâmetros configuráveis na interface gráfica.

Campo	Padrão	Unidade	Descrição
Tempo (T)	0,3	s	Período de amostragem
Referência	26	°C	<i>Setpoint</i> de temperatura
Cooler	0,5	V	Tensão de perturbação
Histerese	0,01	°C	Banda do controlador bang-bang
Entrada MV	6	V	Tensão manual (malha aberta)
K_p	4	—	Ganho proporcional do PID
U_{max}	12	V	Tensão máxima do atuador
T_i	1	s	Constante de tempo integral
U_{min}	0	V	Tensão mínima do atuador
T_d	0,2	s	Constante de tempo derivativa
Polo p	100	—	Polo do filtro derivativo

Fonte: Autor.

Na parte inferior do painel, um componente *listbox* funciona como console de mensagens, registrando eventos da sessão, como a confirmação de conexão serial, envio de parâmetros e eventuais erros de comunicação.

3.1.2.3 Indicadores Numéricos

Quatro painéis de indicadores posicionados na região central superior exibem em tempo real as principais variáveis do processo:

- **Temperatura Atual** — valor instantâneo medido pelo sensor SHT20, em °C;
- **Referência** — *setpoint* vigente enviado ao controlador, em °C;
- **Controle U** — sinal de controle aplicado ao atuador de aquecimento, em V;
- **Cooler RPM** — rotação do cooler calculada pela `TaskRPM` do firmware, em RPM.

Os valores são atualizados a cada ciclo de leitura serial e exibidos com destaque cromático para facilitar a leitura durante os experimentos.

3.1.2.4 Área de Gráficos

A região direita da interface é ocupada por dois eixos (*axes*) sobrepostos verticalmente. O gráfico superior exibe a evolução temporal da temperatura medida $y(t)$ e da referência $r(t)$, permitindo avaliação visual imediata da qualidade da resposta do sistema. O gráfico inferior exibe o sinal de controle $U(t)$, possibilitando a análise da atuação do controlador ao longo do tempo. Ambos os gráficos compartilham o eixo de tempo e são atualizados a cada cinco amostras recebidas, conforme estratégia adotada no script `supervisorio.m` para reduzir o *overhead* de renderização sem comprometer a percepção visual da dinâmica do processo.

O botão **ATUALIZAR GRÁFICOS** força a redesenho imediato das curvas com todos os dados acumulados na sessão, enquanto o botão **LIMPAR** reinicia os buffers e remove as curvas da tela para início de um novo ensaio.

Na versão atual, apresentada como proposta de interface para o TCC-1, os gráficos exibem dados simulados de uma resposta bang-bang gerada numericamente, com o objetivo de demonstrar o layout e o comportamento esperado da interface durante os experimentos reais a serem realizados no TCC-2. A integração completa com a leitura serial em tempo real será implementada na segunda etapa do trabalho, substituindo os dados simulados pelos valores transmitidos pelo Arduino Nano via protocolo CSV descrito na Seção 3.1.2.

3.1.3 Configuração do Ambiente de Desenvolvimento

3.1.3.1 Instalação das Bibliotecas e Dependências

O ambiente de desenvolvimento embarcado utiliza a Arduino IDE como plataforma de compilação e gravação de firmware. Para a execução do *FreeRTOS* no microcontrolador ATmega328P, é necessária a instalação da biblioteca **Arduino_FreeRTOS**, disponível no gerenciador de bibliotecas da IDE sob o nome *FreeRTOS* (autor Phillip Stevens, versão 10.4.3 ou superior). Essa biblioteca provê o *port* do kernel para a arquitetura AVR de 8 bits, incluindo o esquema de alocação dinâmica *Heap 3*, que encapsula as funções `malloc()` e `free()` da biblioteca padrão com suspensão temporária do escalonador para garantir segurança em ambiente multitarefa.

Adicionalmente, é necessária a biblioteca **DFRobot_SHT20** para comunicação com o sensor de temperatura e umidade por meio do protocolo I²C. No lado do computador supervisor, o ambiente utilizado é o MATLAB R2014a, sem necessidade de *toolboxes* adicionais. A comunicação serial é realizada por meio da função nativa `serial()`, disponível desde as primeiras versões da ferramenta. Os *scripts* desenvolvidos utilizam exclusivamente funções nativas do MATLAB R2014a, evitando deliberadamente funções introduzidas em versões posteriores.

3.1.3.2 Configuração da Comunicação Serial Arduino–MATLAB

A comunicação entre o microcontrolador e o computador supervisor é estabelecida pela porta USB do Arduino, que utiliza internamente o conversor FTDI USB-Serial presente na arquitetura da planta (Figura 8). A taxa de transmissão configurada é de 57.600 bits por segundo, valor que equilibra velocidade de transferência e confiabilidade para o comprimento das mensagens adotado.

O protocolo de comunicação adota o formato CSV (*Comma-Separated Values*) com terminador de linha LF (`\n`), escolhido por sua compatibilidade nativa com as funções de leitura serial do MATLAB R2014a. Cada linha transmitida pelo Arduino ao MATLAB contém quatorze campos, conforme descrito no Tabela 2. A recepção no MATLAB é realizada por meio da função `fgetl()`, que retorna uma *string* correspondente a uma linha completa, a qual é então parseada pela sequência `str2double(strsplit(linha, ','))`.

No sentido inverso, o MATLAB envia ao Arduino uma linha CSV com doze campos para atualização dos parâmetros de controle em tempo real, no formato:

```
state, t, ref, mv, umax, umin, h, pertub, k, ti, td, polo
```

Tabela 2: Campos do protocolo serial CSV (Arduino → MATLAB).

Campo	Descrição	Unidade
<code>time</code>	Tempo acumulado de execução	s
<code>y</code>	Temperatura medida pelo sensor SHT20	°C
<code>U</code>	Sinal de controle aplicado ao atuador	V
<code>state</code>	Modo de operação (1=MA, 2=BB, 3=PID)	—
<code>ref</code>	Valor de referência (<i>setpoint</i>)	°C
<code>K</code> , <code>Ti</code> , <code>Td</code> , <code>p</code>	Parâmetros do controlador PID	—
<code>Umax</code> , <code>Umin</code>	Limites de saturação do sinal de controle	V
<code>h</code>	Banda de histerese do controlador bang-bang	°C
<code>pertub</code>	Tensão aplicada ao cooler (perturbação)	V
<code>rpm</code>	Rotação do cooler medida pelo sensor Hall	RPM

Fonte: Autor.

onde `t` é o período de amostragem em segundos. A recepção no Arduino é realizada na `TaskSerial` por meio de leitura byte a byte do buffer serial, com acumulação em vetor de caracteres e processamento ao receber o terminador `\n`. A função `sscanf()` realiza o *parse* dos campos e os valores são protegidos por *mutex* antes de serem escritos nas variáveis globais compartilhadas.

3.1.4 Projeto da Arquitetura de Software Embarcado

A arquitetura de software embarcado organiza o sistema em um conjunto de tarefas independentes gerenciadas pelo kernel do FreeRTOS, garantindo o determinismo temporal exigido em sistemas de controle em tempo real e isolando funcionalmente as responsabilidades do sistema.

3.1.4.1 Definição de Tarefas e Prioridades

O firmware é estruturado em quatro tarefas concorrentes, cujas responsabilidades, prioridades e períodos de execução estão descritos no Quadro 3. A atribuição de prioridades segue o princípio de que tarefas críticas ao desempenho do sistema de controle recebem prioridade superior às tarefas de suporte.

A `TaskSensor` e a `TaskControl` operam com prioridade 3 e período idêntico T , configurável em tempo real via serial. Ambas utilizam `vTaskDelayUntil()` para execução periódica com referência absoluta de tempo.

A `TaskRPM` opera com prioridade 2 e período fixo de 1000 ms, calculando a rotação do cooler a cada segundo.

A `TaskSerial` opera com prioridade 1 para evitar interferência sobre o laço de controle, sendo executada após a conclusão das tarefas de maior prioridade em cada ciclo.

O tamanho de pilha de cada tarefa foi definido em 150 *words* (300 bytes), resultando em consumo total estimado de aproximadamente 1710 bytes de SRAM, dentro dos 2 KB disponíveis no ATmega328P, conforme detalhado na Tabela 4.

Tabela 3: Definição das tarefas FreeRTOS e níveis de prioridade.

Tarefa	Prioridade	Período	Responsabilidade
TaskSensor	3 (alta)	T ms	Leitura periódica de temperatura via I ² C (SHT20). Usa <code>vTaskDelayUntil()</code> para periodicidade com referência absoluta de tempo.
TaskControl	3 (alta)	T ms	Execução do algoritmo de controle selecionado (PID, bang-bang ou malha aberta). Acessa dados do sensor via <i>mutex</i> .
TaskRPM	2 (média)	1000 ms	Cálculo do RPM do cooler por contagem de pulsos do sensor Hall. Proteção via seção crítica (<code>taskENTER_CRITICAL</code>).
TaskSerial	1 (baixa)	T ms	Envio de dados ao MATLAB em formato CSV e recepção de novos parâmetros de controle via serial.

Fonte: Autor.

Tabela 4: Estimativa de uso de SRAM (ATmega328P — 2 KB total).

Item	Bytes
Kernel FreeRTOS	≈ 250
4 pilhas de tarefa (150 <i>words</i> cada)	≈ 1200
Variáveis globais de controle	≈ 80
Buffers de comunicação serial	≈ 180
Total estimado	≈ 1710
Margem disponível	≈ 310

Fonte: Autor.

3.1.4.2 Estratégia de Escalonamento Adotada

O escalonador do FreeRTOS é configurado no modo preemptivo com fatiamento de tempo (*time slicing*) ativo. Como a `TaskSensor` e a `TaskControl` possuem a mesma prioridade e período, o escalonador as intercala por fatiamento de tempo a cada *tick* do sistema.

A `TaskSensor` recebe o processador primeiro por ter sido criada antes no `setup()`, garantindo que a leitura do sensor sempre preceda o cálculo do sinal de controle dentro do mesmo período de amostragem.

O acesso às variáveis compartilhadas entre as tarefas é protegido por um *mutex* global (`xMutex`), criado no `setup()` antes da inicialização do escalonador.

Cada tarefa solicita o *mutex* por no máximo 10 ms antes de prosseguir, evitando bloqueio indefinido em caso de contenção.

A exceção é a variável de contagem de pulsos do tacômetro (`tachCount`), cujo acesso é protegido por seção crítica (`taskENTER_CRITICAL`/`taskEXIT_CRITICAL`), mecanismo mais adequado para variáveis acessadas em rotinas de interrupção.

3.1.5 Implementação dos Algoritmos de Controle

Os algoritmos de controle são executados integralmente na `TaskControl`, que opera com alta prioridade e periodicidade determinística garantida pelo FreeRTOS.

A seleção do algoritmo ativo é feita pela variável `state`, configurável em tempo real via serial, permitindo alternar entre os modos de operação sem necessidade de recompilação ou *reset* do sistema.

4 PROPOSTA DE TRABALHO

4.1 ESCOPO E DELIMITAÇÕES

5 CRONOGRAMA DE ATIVIDADES

O cronograma de atividades previsto para a conclusão deste trabalho está apresentado no 1. As atividades estão distribuídas ao longo do segundo semestre de 2026, contemplando a finalização da implementação, a realização dos experimentos, a análise dos resultados e a redação do documento final.

Atividade	Ago	Set	Out	Nov	Dez
Revisão bibliográfica complementar	•	•			
Sintonia experimental do controlador PID	•	•			
Validação experimental (resposta ao degrau)		•	•		
Validação (rejeição a distúrbios)			•		
Análise comparativa PID vs. bang-bang			•	•	
Redação dos capítulos de resultados			•	•	
Redação das considerações finais				•	•
Revisão geral e formatação				•	•
Entrega e defesa do TCC					•

Fonte: Autor.

Quadro 1: Cronograma de atividades para o segundo semestre de 2026.

REFERÊNCIAS

- AMAZON WEB SERVICES. *FreeRTOS: Real-time operating system for microcontrollers and small microprocessors*. United States, 2024. Disponível em: <https://www.freertos.org/>. Acesso em: 4 maio 2026.
- BARRY, R. *Mastering the FreeRTOS™ Real Time Kernel: A Hands-On Tutorial Guide*. [S. l.], 2016. Disponível em: https://www.freertos.org/fr-content-src/uploads/2018/07/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf. Acesso em: 4 abr. 2026.
- BAZANELLA, A. S.; SILVA JR., J. M. G. DA. *Sistemas de Controle: Princípios e Métodos de Projeto*. [S. l.]: UFRGS, 2005.
- DENARDIN, G. W.; BARRIQUELLO, C. H. *Sistemas operacionais de tempo real e sua aplicação em sistemas embarcados*. São Paulo, Brasil: Editora Edgard Blücher Ltda., 2019. ISBN 978-85-212-1396-3. Disponível em: https://storage.blucher.com.br/book/pdf_preview/9788521213963-amostra.pdf. Acesso em: 4 abr. 2026.
- JÚNIOR, S. L. S.; SILVA, R. A. *Automação e Instrumentação Industrial com Arduino: Teoria e Projetos*. Rio de Janeiro: Érica, 2015.
- LAPLANTE, P. A. *Real-Time Systems Design and Analysis: An Engineer's Handbook*. 4. ed. [S. l.]: Wiley, 2004.
- MORAIS, J. *RTOS: Um ambiente multi-tarefas para sistemas embarcados*. [S. l.: s. n.], 2018. <https://embarcados.com.br/rtos-sistema-operacional-de-tempo-real/>. Acesso em: 17 jul. 2024.
- NISE, N. S. *Engenharia de Sistemas de Controle*. 7. ed. Rio de Janeiro: LTC, 2017. ISBN 9788521634379.
- OGATA, K. *Modern Control Engineering*. 5. ed. Upper Saddle River, NJ: Pearson, 2010. ISBN 9780136156734.
- SPOHN, M. A.; NETO, F. C.; FASSINI, L. T. *Uma avaliação do sistema operacional FreeRTOS na plataforma Arduino Uno*. Chapecó, SC, Brasil, 2019. p. 12. Disponível em: <https://revistas.unifacs.br/index.php/rsc/article/view/6016>. Acesso em: 3 abr. 2026.

THE MATHWORKS, INC. *MATLAB*. Natick, Massachusetts, USA. s.d. Disponível em:
<https://matlab.mathworks.com/>. Acesso em: 5 abr. 2026.